Programmation procédurale (UAA11)

Atelier : les chaînes de caractères

Objectifs de l'atelier :

- ✔ Découvrir ce qu'est réellement une chaîne en mémoire.
- ✔ Distinguer la place réservée et la longueur du texte.
- ✔ Comparer les deux façons de saisir une chaîne.
- ✔ Copier, assembler et comparer des chaînes.
- ✔ Mesurer le danger d'une chaîne trop longue pour son tableau.

Avant de commencer

Cet atelier se fait seul, avant la séance de théorie. Tu n'as rien à apprendre par cœur : tu vas compiler de petits programmes, les exécuter, les modifier, et découvrir par toi-même comment le C se comporte. Les règles que tu formuleras serviront de point de départ à la séance.

Les programmes à télécharger :

https://lucasembrechts.github.io/UAA11/fichiers/atelier_chaines.zip

Comment travailler :

- ➢ décompresse l'archive : tu obtiens un fichier `.c` par programme. Le nom du fichier est indiqué au-dessus de chaque programme ;
- ➢ ouvre chaque fichier dans ton éditeur habituel, compile-le et exécute-le ;
- ➢ **avant** chaque exécution, écris ta prédiction. Ne la corrige pas après coup : une prédiction fausse est justement ce qui va t'apprendre quelque chose ;
- ➢ certains programmes ne compilent pas : c'est voulu. Lis alors attentivement le message du compilateur, il fait partie de la découverte ;
- ➢ ne cherche pas la réponse dans le cours ou sur Internet : l'atelier est conçu pour que tu la trouves en observant ;
- ➢ à la fin de chaque section, résume ta découverte dans le cadre « Ma règle ». L'ensemble de tes règles formera ta fiche pour la séance théorique.

Section 0 : une chaîne, c'est quoi ?

Ce programme affiche le code (le numéro dans la table ASCII) contenu dans chaque case de la variable `ville`.

Fichier : section0.c

```
#include <stdio.h>
#include <string.h>

void main(void) {
   char ville[] = "Namur";
   for (int i = 0; i < 6; i++) {
      printf("case %d : code %d\n", i, ville[i]);
   }
   printf("sizeof : %d\n", sizeof(ville));
   printf("strlen : %d\n", strlen(ville));
}
```

Prédis

Le mot « Namur » compte 5 lettres, mais la boucle affiche 6 cases. Que contiendra, selon toi, la sixième case (case 5) ? Que vaudront `sizeof` et `strlen` ?

Exécute, puis explique

Que contient la case 5 ? Pourquoi cette case existe-t-elle, alors que « Namur » n'a que 5 lettres ?

Retrouve dans une table ASCII le caractère qui correspond au code 78.

Quelle différence fais-tu entre ce que mesurent `sizeof` et `strlen` ?

Modifie

Remplace `char ville[] = "Namur";` par `char ville[20] = "Namur";`. Prédis les deux dernières lignes, puis exécute.

✎ Ma règle : qu'est-ce qu'une chaîne de caractères en C ? Que mesurent sizeof et strlen ?

Section 1 : le '\0' décide de tout

Fichier : section1.c

```
#include <stdio.h>
#include <string.h>

void main(void) {
   char mot[] = "Bonjour";
   mot[3] = '\0';
   printf("%s\n", mot);
   printf("strlen : %d / sizeof : %d\n", strlen(mot), sizeof(mot));
   mot[3] = 'j';
   printf("%s\n", mot);
}
```

Prédis

Qu'affiche ce programme ?

Exécute, puis explique

Pourquoi le premier affichage s'arrête-t-il à « Bon » ?

`strlen` a changé, mais pas `sizeof`. Pourquoi ?

Le dernier affichage redonne « Bonjour ». Qu'est-ce que cela prouve au sujet des lettres « jour » ?

Modifie

Ajoute à la fin du programme `printf("%s\n", mot + 3);`, puis `mot[0] = 'b';` et `printf("%s\n", mot);`. Prédis, puis exécute.

✎ Ma règle : comment printf sait-il où une chaîne s'arrête ? Comment couper une chaîne ?

Section 2 : saisir une chaîne

Fichier : section2.c

```
#include <stdio.h>
#include <string.h>

void main(void) {
   char nom[50];
   printf("Ton nom complet : ");
   scanf("%s", nom);
   getchar();
   printf("Bonjour %s !\n", nom);
   printf("Longueur : %d\n", strlen(nom));
}
```

Prédis

Tu vas exécuter ce programme en tapant exactement `Jean Dupont`. Prédis ce qu'il affichera.

Exécute, puis explique

Où est passé « Dupont » ?

Dans `scanf("%s", nom)`, il n'y a pas de `&` devant `nom`, contrairement à `scanf("%d", &age)`. Essaie de deviner pourquoi.

Modifie

Remplace les deux lignes `scanf("%s", nom);` et `getchar();` par `fgets(nom, 50, stdin);`. Exécute à nouveau avec `Jean Dupont`. Qu'observes-tu ?

Ajoute juste après le `fgets` la ligne `nom[strlen(nom) - 1] = '\0';`. Exécute. Explique ce que fait cette ligne.

À quoi sert, selon toi, le `50` dans `fgets(nom, 50, stdin)` ?

✎ Ma règle : quelle différence entre scanf("%s") et fgets ?

Section 3 : copier et assembler

Fichier : section3.c

```
#include <stdio.h>
#include <string.h>

void main(void) {
   char a[20] = "lune";
   char b[20];
   b = a;
   printf("a : %s / b : %s\n", a, b);
}
```

Prédis

Ce programme compile-t-il ? Si oui, qu'affiche-t-il ?

Compile, puis explique

Que dit le message du compilateur ? Traduis-le avec tes mots.

Voici la bonne façon de copier :

Fichier : section3_copie.c

```
#include <stdio.h>
#include <string.h>

void main(void) {
   char a[20] = "lune";
   char b[20];
   strcpy(b, a);
   b[0] = 'd';
   printf("a : %s / b : %s\n", a, b);
}
```

Prédis l'affichage, puis exécute. Après la copie, modifier `b` a-t-il changé `a` ? Que fait donc `strcpy` ?

Et pour assembler deux chaînes :

Fichier : section3_assembler.c

```
#include <stdio.h>
#include <string.h>

void main(void) {
   char prenom[20] = "Lina";
   char nom[20] = "Martin";
   char complet[40] = "";
   strcat(complet, prenom);
   strcat(complet, nom);
   printf("[%s]\n", complet);
   printf("Longueur : %d\n", strlen(complet));
}
```

Prédis l'affichage, puis exécute. Où est passé l'espace entre le prénom et le nom ?

Modifie

Corrige le programme pour obtenir « Lina Martin ». Quelle longueur obtiens-tu ?

Remplace `char complet[40] = "";` par `char complet[40];` et exécute plusieurs fois. Qu'observes-tu ? Pourquoi ?

`complet` compte 40 cases. Combien de cases faut-il au minimum pour « Lina Martin » ?

✎ Ma règle : comment copie-t-on et assemble-t-on des chaînes ?

Section 4 : comparer deux chaînes

Fichier : section4.c

```
#include <stdio.h>
#include <string.h>

void main(void) {
   char a[20] = "pomme";
   char b[20] = "pomme";
   if (a == b) {
      printf("avec == : identiques\n");
   } else {
      printf("avec == : differentes\n");
   }
   if (strcmp(a, b) == 0) {
      printf("avec strcmp : identiques\n");
   } else {
      printf("avec strcmp : differentes\n");
   }
}
```

Prédis

Les deux variables contiennent « pomme ». Qu'afficheront les deux tests ?

Exécute, puis explique

Le compilateur t'a-t-il averti de quelque chose ? Ajoute `printf("%p %p\n", a, b);` et exécute : que compare donc `==` entre deux chaînes ?

`strcmp` renvoie 0 quand les chaînes sont identiques. Pourquoi faut-il écrire `if (strcmp(a, b) == 0)` et pas simplement `if (strcmp(a, b))` ?

Que renvoie `strcmp` quand les chaînes sont différentes ?

Fichier : section4_ordre.c

```
#include <stdio.h>
#include <string.h>

void main(void) {
   printf("%d\n", strcmp("pomme", "poire"));
   printf("%d\n", strcmp("poire", "pomme"));
   printf("%d\n", strcmp("Pomme", "pomme"));
   printf("%d\n", strcmp("pomme", "pommes"));
}
```

Pour chaque ligne, prédis le **signe** du nombre affiché : négatif, nul ou positif. La valeur exacte dépend de l'ordinateur, seul le signe compte.

D'après tes observations, sur quoi `strcmp` se base-t-il pour classer deux chaînes ? « Pomme » viendrait-il avant « abricot » ?

✎ Ma règle : comment compare-t-on deux chaînes ?

Section 5 : une chaîne trop longue

Fichier : section5.c

```
#include <stdio.h>
#include <string.h>

void main(void) {
   char code[8] = "ABC";
   char petit[4];
   strcpy(petit, "123456789");
   printf("petit : %s\n", petit);
   printf("code  : %s\n", code);
}
```

Prédis

La variable `petit` ne compte que 4 cases. Le programme va-t-il compiler ? S'exécuter ? Qu'affichera-t-il ?

Exécute et observe

Le compilateur a-t-il affiché un avertissement ? Le programme s'est-il arrêté ? Qu'affiche-t-il ? Compare avec ton voisin.

Explique

Combien de caractères `strcpy` a-t-il essayé d'écrire dans `petit`, '\0' compris ? Où sont allés ceux qui ne tenaient pas ?

`strcpy` a-t-il vérifié la taille de `petit` ? Qui doit le faire ?

✎ Ma règle : avant un strcpy ou un strcat, quelle question faut-il toujours se poser ?

Défi final

Crée un fichier `defi.c`. Le programme construit l'adresse e-mail d'un élève sous la forme `prenom.nom@ecole.be`, puis lui demande de la retaper pour la confirmer.

- le prénom et le nom sont saisis avec `fgets` (les noms composés doivent être acceptés), sans le `\n` final ;
- l'adresse est construite dans une seule variable, avec `strcpy` et `strcat` ;
- choisis la taille de chaque tableau et justifie-la ;
- le programme indique si l'adresse retapée correspond.

Exemple d'exécution (les réponses de l'utilisateur suivent les deux-points) :

```
Prenom : Lina
Nom : Van Damme
Ton adresse : Lina.Van Damme@ecole.be (23 caracteres)
Retape ton adresse : Lina.Van Damme@ecole.be
Adresse confirmee.
```

Si le prénom et le nom comptent chacun au plus 29 caractères, quelle est la taille minimale de la variable qui contient l'adresse ?